

also verify that the next `Link`'s key is less than the `goal` key; otherwise, the loop can end.

Listing 5-14 *The `insert()` and `delete()` Methods of `OrderedList`*

```
class OrderedList(LinkedList):
...

    def insert(self, newDatum): # Insert a new datum based on key order
        goal = self.__key(newDatum) # Get target key
        previous = self # Link or OrderedList before goal Link
        while (previous.getNext() is not None and # Has next link and
                self.__key(previous.getNext().getData())
                < goal): # next link's key is before the goal
            previous = previous.getNext() # Advance to next link
        newLink = Link( # Build a new Link node with new
            newDatum, previous.getNext()) # datum and remainder of list
        previous.setNext(newLink) # Update previous' first/next pointer

    def delete(self, goal): # Delete first Link with matching key
        if self.isEmpty(): # Empty list? Raise an exception
            raise Exception("Cannot delete from empty linked list")

        previous = self # Link or OrderedList before Link
        while (previous.getNext() is not None and # Has next link and
                self.__key(previous.getNext().getData())
                < goal): # next link's key is before the goal
            previous = previous.getNext() # Advance to next link
        if (previous.getNext() is None or # If goal key not in next
            goal != # Link after previous
            self.__key(previous.getNext().getData())):
            raise Exception("No datum with matching key found in list")

        toDelete = previous.getNext() # Store Link to delete
        previous.setNext(toDelete.getNext()) # Remove it from list

        return toDelete.getData() # Return data in deleted Link
```

After the loop ends, `insert()` constructs the new `Link` with the provided `newDatum` and the rest of the list after the `previous` pointer. The `__next` pointer of `previous` is updated similarly to the other linked list classes. By inheriting the definition of `setNext()` as a synonym for the `setFirst()` method, the

`OrderedList.insert()` method updates either the `__first` pointer or the `__next` pointer depending on the data type of `previous`.

The `delete()` method uses a similar structure as the `insert()` method, after first checking for an empty list and throwing an exception if one is found. It uses the same style of `while` loop to get `previous` to point at the `Link` immediately before the link where the `goal` key is or should be. To avoid having the loop end with `previous` pointing to a position after where the `goal` key would go, the loop condition looks ahead at the next link's key, if there is one. When the loop is done, it checks if the `Link` just after `previous` has the `goal` key and raises an exception if it doesn't. Having verified that the `goal` key was found, it can note the link `toDelete` and remove it by modifying the appropriate `__next` or `__first` field by calling `setNext()` on the previous link. It replaces the old value by calling `getNext()` on the `Link` to be deleted. The data stored in the `toDelete` link is returned.

The movement of the `previous` pointer and the addition/removal of the next link is animated in the Insert/Delete operations of the Ordered List Visualization tool. Look at the individual steps to see the operations in detail.

[Listing 5-15](#) shows a client program to test the `OrderedList` class. It starts with an empty ordered list using the default sorting key function, `identity()`. It inserts an alternating sequence of positive and negative integers. It displays the ordered list and then searches it for certain values to see what the `find()` and `search()` methods return. Finally, it removes the numbers in the order they were inserted, which means it first deletes items from the middle of the list and finishes by deleting the ones on the ends.

Listing 5-15 *The `OrderedListClient.py` Program*

```
from OrderedList import *

olist = OrderedList()
print('Initial list has', len(olist), 'element(s) and empty =',
      olist.isEmpty())

for i in range(5):
    olist.insert((-1 - i) ** i)
print('After inserting', len(olist),
      'numbers into the ordered list, it contains:\n', olist,
      'and empty =', olist.isEmpty())
```

```

for value in [9, 999]:
    for sign in [-1, 1]:
        val = sign * value
        print('Trying to find', val, 'in ordered list returns',
              olist.find(val), ', search returns', olist.search(val))

print('Deleting items from the ordered list:')
for i in range(5):
    number = (-1 - i) ** i
    print('After deleting', olist.delete(number),
          'the list is', olist)

```

The result of running this program is

```

$ python3 OrderedListClient.py
Initial list has 0 element(s) and empty = True
After inserting 5 numbers into the ordered list, it contains:
[-64 > -2 > 1 > 9 > 625] and empty = False
Trying to find -9 in ordered list returns -2 , search returns None
Trying to find 9 in ordered list returns 9 , search returns 9
Trying to find -999 in ordered list returns -64 , search returns None
Trying to find 999 in ordered list returns None , search returns None
Deleting items from the ordered list:
After deleting 1 the list is [-64 > -2 > 9 > 625]
After deleting -2 the list is [-64 > 9 > 625]
After deleting 9 the list is [-64 > 625]
After deleting -64 the list is [625]
After deleting 625 the list is []

```

Notice the result of the call to `find(-9)`. It returns a pointer to the `Link` holding `-2`. That might seem odd, but `-9` falls between `-64` and `-2`. The `find()` method returns a pointer to a link immediately after where the given key should be inserted. The call to `find(9)` does find the `Link` holding `9` in it, and that is the only call to the `search()` method that returns something other than `None`. The calls to `find -999` and `999` show the results when the goal key would land before or after all the list items, respectively.

Efficiency of Ordered Linked Lists

Insertion and deletion of arbitrary items in the ordered linked list require $O(N)$ comparisons ($N/2$ on the average) because the appropriate location must be found by stepping through the list. The minimum value, however, can be found, or deleted, in $O(1)$ time because it's at the beginning of the list. If an

application frequently accesses the minimum item, and fast insertion isn't critical, then an ordered linked list is an effective choice. Similarly, if the maximum item is needed much more frequently than the minimum and the same $O(N)$ average insertion time is acceptable, the items could be ordered in descending order. If both the minimum and maximum were needed, a double-ended ordered list would be good. A priority queue might be implemented by a double-ended ordered linked list, for example.

List Insertion Sort

An ordered list can be used as a fairly efficient sorting mechanism for data in an array. Suppose you have an array of unordered data items. If you take the items from the array and insert them one by one into the ordered list, they'll be placed in order automatically. If you then remove them from the list, always deleting from the front and putting them back in the array by increasing indices, the array will be ordered.

This type of sort turns out to be substantially more efficient than the more usual insertion sort within an array, described in [Chapter 3](#), “[Simple Sorting](#),” because fewer copies are necessary. It's still an $O(N^2)$ process because inserting each item into the ordered list involves comparing a new item with an average of half the items already in the list. There are N items to insert, which results in about $N^2/4$ comparisons. Each item is copied only twice, however—once from the array to the list and once from the list to the array. The $2 \times N$ copies compare favorably with the insertion sort within an array, where there are about N^2 copies.

The downside of the list insertion sort, compared with an array-based insertion sort, is that it takes somewhat more than twice as much memory: the array and linked list must be in memory at the same time. If you have an ordered linked list class handy, however, the list insertion sort is a convenient way to sort arrays that aren't too large.

Doubly Linked Lists

Let's examine another variation on the linked list: the **doubly linked list** (not to be confused with the *double-ended list*). What's the advantage of a doubly linked list? A potential problem with ordinary linked lists is that it's difficult to traverse backward along the list. A statement like

```
current = current.getNext()
```

steps conveniently to the next link, but there's no corresponding way to return to the previous link. Depending on the application, this limitation could pose problems.

For example, imagine a text editor in which a linked list is used to store the text. Each text line on the screen is stored as a `String` object referenced from a link. When the user moves the cursor downward on the screen, the program steps to the next link to manipulate or display the new line. What happens if the user moves the cursor upward? In an ordinary linked list, you would need to return `current` (or its equivalent) to the start of the list and then step all the way down again to the new current link. This isn't very efficient. You want to make a single step upward.

The doubly linked list provides this capability. It allows programs to traverse backward as well as forward through the list. The secret is that each link keeps two references to other links instead of one. The first is to the next link, as in ordinary lists. The second is to the previous link. [Figure 5-10](#) shows a double-ended version of this type of list.

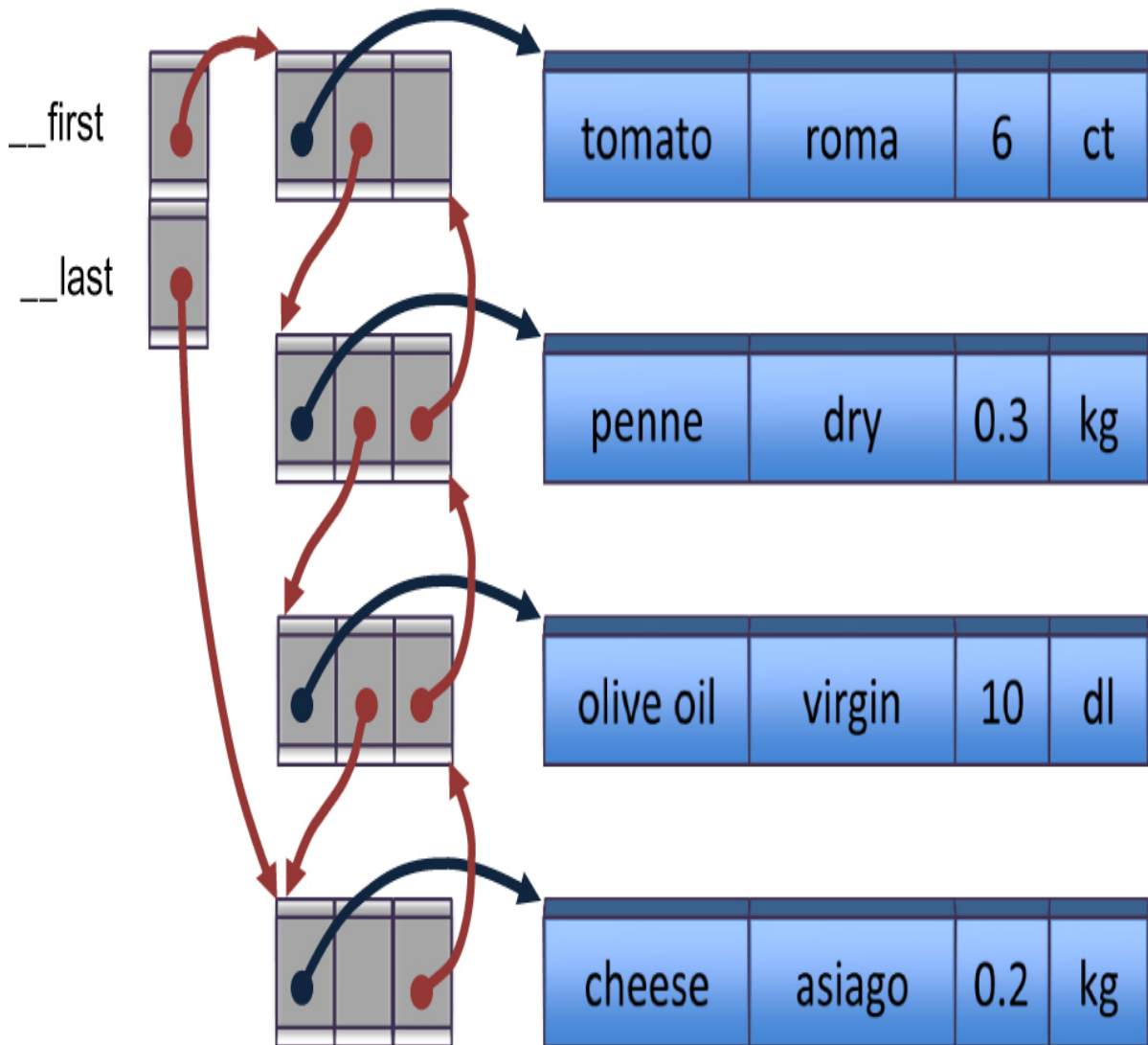


Figure 5-10 *A doubly linked (and double-ended) list*

The chain of links ends with a null pointer (an empty box in the figure and a value of `None` in Python) for both the forward and reverse chains. The downside of doubly linked lists is that every time you insert or delete a link, you must deal with four links instead of two—two attachments to the previous link and two attachments to the following one. That will become clearer when we look at the implementation a little later. It should be no surprise that each link record is a little bigger because of the extra reference needed for the reverse link.

Let's look at how to manage the extra linkages. The specification for the `Link` class in a doubly linked list (as a subclass of singly linked list) is shown in [Listing 5-16](#).

Listing 5-16 *The Link Class of the DoublyLinkedList.py Module*

```
import LinkedList

class Link(LinkedList.Link):
    # One datum in a linked list
    def __init__(self, datum,
                  next=None,
                  previous=None):
        # Constructor with datum
        # and optional next and
        # previous pointers
        self.__data = datum
        self.__next = next
        self.__previous = previous
        # reference to next item in list
        # reference to previous item

    def getData(self): return self.__data
    # Accessors
    def getNext(self): return self.__next
    def getPrevious(self): return self.__previous
    def setData(self, d): self.__data = d
    def setNext(self, link):
        # Accessor that enforces type
        if link is None or isinstance(link, Link):
            self.__next = link
        else:
            raise Exception("Next link must be Link or None")
    def setPrevious(self, link):
        # Accessor that enforces type
        if link is None or isinstance(link, Link):
            self.__previous = link
        else:
            raise Exception("Previous link must be Link or None")

    def isFirst(self): return self.__previous is None
```

The definition for this `Link` class is only a little more complex than that for the singly linked list shown in [Listing 5-2](#). The class definition makes the doubly linked version a subclass of the singly linked version and adds a private `__previous` field, which means that new accessor methods need to be defined for getting and setting its value. Just like for the `__next` pointer, the `setPrevious()` method enforces the constraint about the type of reference stored in that field. Most of the methods need to be rewritten in the subclass to correctly access the private fields in Python. Only the `isLast()` and `__str__()` methods remain the same in both `Link` classes. A new method for testing whether this is the first `Link` in a list, `isFirst()`, is now easy with the addition of the `__previous` field.

A doubly linked list doesn't necessarily need to be a double-ended list (keeping a reference to the last element on the list), but creating it this way is useful. The implementation of `DoublyLinkedList` shown in [Listing 5-17](#) uses both a `__first` and `__last` pointer to make it easy to traverse the list in reverse order. It has the usual accessor methods for getting and setting the fields. The methods for setting the pointers enforce the type constraint on those fields and perform special checks to update both ends of the list when adding the first or deleting the last link.

Listing 5-17 *The `DoublyLinkedList` Class Constructor and Accessors*

```
class DoublyLinkedList(LinkedList.LinkedList):
    def __init__(self):                # Constructor
        self.__first, self.__last = None, None

    def getFirst(self): return self.__first # Accessors
    def getLast(self): return self.__last

    def setFirst(self, link):          # Set first link
        if link is None or isinstance(link, Link): # Check type
            self.__first = link
            if (self.__last is None or          # If list was empty or
                link is None):                  # list is being truncated
                self.__last = link              # update both ends
        else:
            raise Exception("First link must be Link or None")

    def setLast(self, link):           # Set last link
        if link is None or isinstance(link, Link): # Check type
            self.__last = link
            if (self.__first is None or          # If list was empty or
                link is None):                  # list is being truncated
                self.__first = link              # update both ends
        else:
            raise Exception("Last link must be Link or None")

    def traverseBackwards(             # Apply a function to all Links in list
        self, func=print):            # backwards from last to first
        link = self.getLast()          # Start with last link
        while link is not None:        # Keep going until no more links
            func(link)                 # Apply the function to the link
            link = link.getPrevious()   # Move on to previous link
```

The `DoublyLinkedList` class inherits the methods defined for the `LinkedList` class to traverse the list in the forward direction. More specifically, the `isEmpty()`, `first()`, `traverse()`, `__len__()`, and `__str__()` methods of `LinkedList` all work without modification. A new `traverseBackwards()` method applies a function to each list item in the reverse direction.

Insertion and Deletion at the Ends

The next methods to look at are the ones for inserting and deleting links at the beginning and end of the doubly linked list. The `insertFirst()` method inserts at the beginning of the list, and `insertLast()` inserts at the end. Similarly, the `deleteFirst()` and `deleteLast()` methods delete the first and last `Links` in the list. They are shown in [Listing 5-18](#). Later we look at methods to insert and delete links in the middle.

Listing 5-18 *Methods to Insert and Delete the Ends of `DoublyLinkedLists`*

```
class DoublyLinkedList(LinkedList.LinkedList):
... (other definitions shown before) ...

    def insertFirst(self, datum): # Insert a new datum at start of list
        link = Link(datum,          # New link has datum
                     next=self.getFirst()) # and precedes current first
        if self.isEmpty():          # If list is empty,
            self.setLast(link)      # insert link as last (and first)
        else:                       # Otherwise, first Link in list
            self.getFirst().setPrevious(link) # now has new Link before
            self.setFirst(link)      # Update first link

    insert = insertFirst           # Override parent class insert

    def insertLast(self, datum): # Insert a new datum at end of list
        link = Link(datum,          # New link has datum
                     previous=self.getLast()) # and follows current last
        if self.isEmpty():          # If list is empty,
            self.setFirst(link)     # insert link as first (and last)
        else:                       # Otherwise, last Link in list
            self.getLast().setNext(link) # now has new Link after
            self.setLast(link)      # Update last link

    def deleteFirst(self):         # Delete and return first link's data
        if self.isEmpty():        # If list is empty, raise exception
```

```

        raise Exception("Cannot delete first of empty list")
    first = self.getFirst()      # Store the first link
    self.setFirst(first.getNext()) # Remove first, advance to next
    if self.getFirst():          # If that leaves a link in the list,
        self.getFirst().setPrevious(None) # Update its predecessor
    return first.getData()       # Return data from first link

def deleteLast(self):          # Delete and return last link's data
    if self.isEmpty():         # If list is empty, raise exception
        raise Exception("Cannot delete last of empty list")
    last = self.getLast()      # Store the last link
    self.setLast(last.getPrevious()) # Remove last, advance to prev
    if self.getLast():         # If that leaves a link in the list,
        self.getLast().setNext(None) # Update its successor
    return last.getData()       # Return data from last link

```

The `insertFirst()` method first creates the new `Link` holding the new datum. The current first `Link` of the list is passed as the next link to follow the new link. The `__previous` field of the new link is left empty because this will be the first `Link` of the list. The next step is to update the `__first` (and possibly the `__last` field) to be the new link. There are two cases:

- If the list is empty, the method updates both with a single call to `setLast()`.
- If there are already some links, then it must change the reverse pointer from the current first link to point back at the new link. Then it replaces the first link with the newly created `Link`.

The process for inserting the string "a" into a doubly linked list containing ["A", "B"] is shown in [Figure 5-11](#).

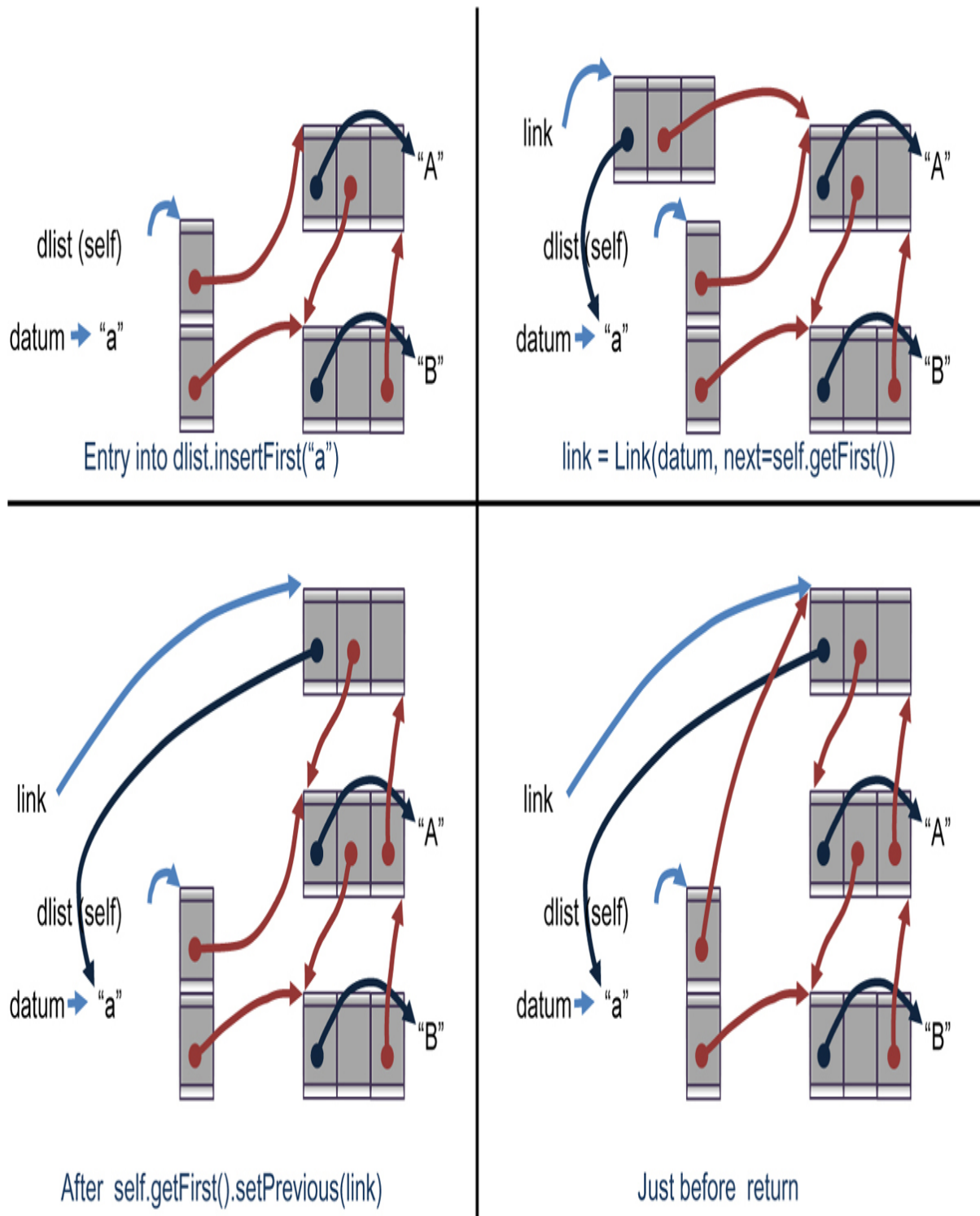


Figure 5-11 Insertion at the beginning of a doubly linked list

After defining the `insertFirst()` method, [Listing 5-18](#) also redefines `insert()` to be synonymous with it. That redefinition is needed so that the

simpler name in the parent class, `LinkedList`, refers to the same operation. The `insertLast()` follows the same structure but updates the `__last` field. It's something like a mirror image of `insertFirst()`, swapping `first` $\rightleftharpoons$ `last`, and `previous` $\rightleftharpoons$ `next`.

Deleting a `Link` from one of the ends of the list also involves carefully updating the pointers. The order of the updates is important so that you avoid removing a reference that is needed later. The implementation of `deleteFirst()` checks whether the list is empty and throws an exception if it is. If not, it stores a pointer to the first `Link`. That `first` pointer is used to get the data in that `Link` after it's been removed from the list. Then it advances the `__first` field to point at the second `Link`. The pointer to that second `Link` might be `None`. If it is, the `setFirst()` method also updates the `__last` field to be `None`. Otherwise, there is at least one more link in the list, and its `__previous` field must be updated to be `None` because it just became the first `Link`, as shown in [Figure 5-12](#). Finally, the data from the deleted `Link` is returned.

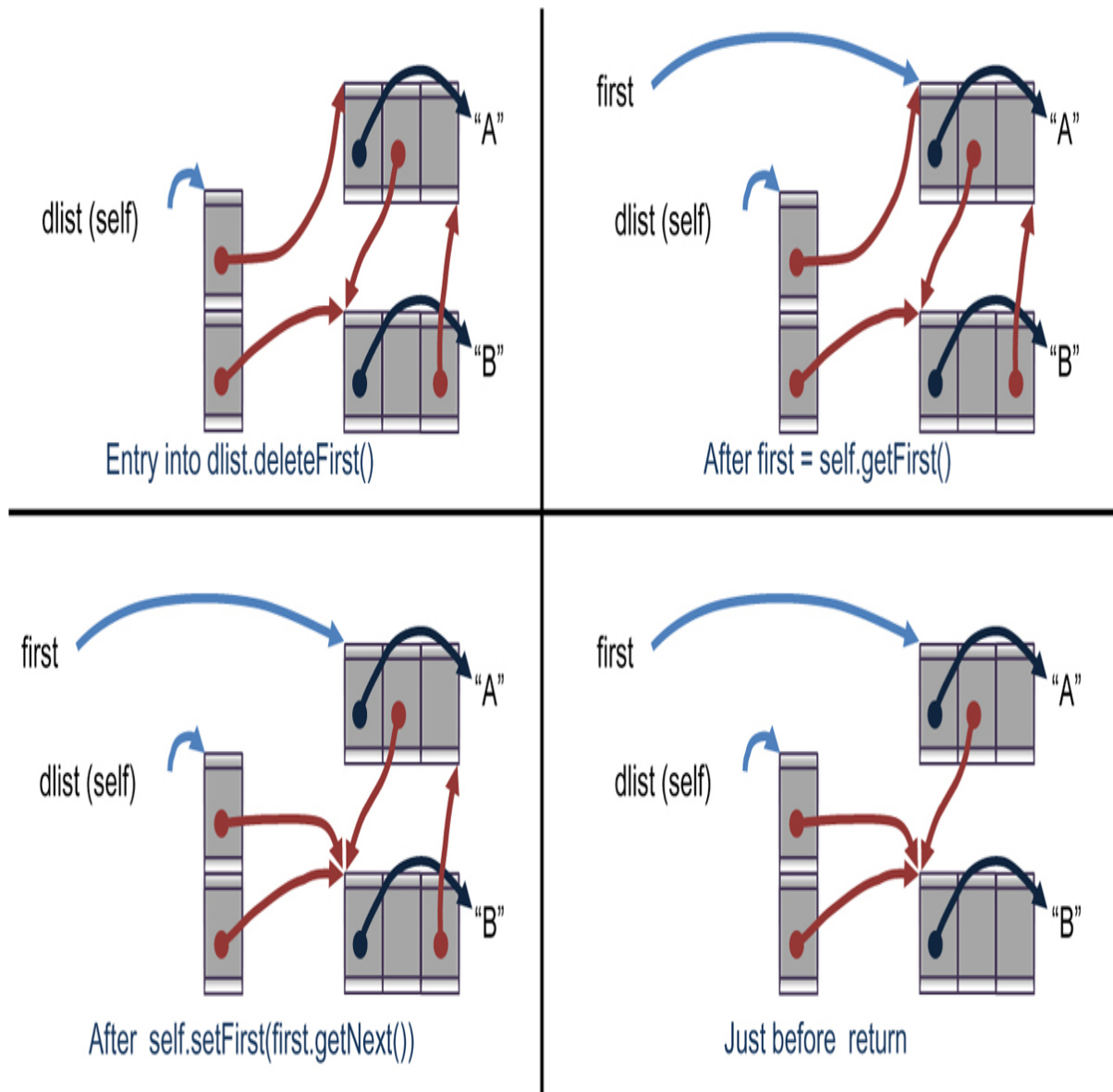


Figure 5-12 *Deleting the first item of a doubly linked list*

Insertion and Deletion in the Middle

Modifying `Links` in the middle of a doubly linked list is slightly more complicated because all four pointers need to be considered (the forward-reverse pointers both before and after). To specify where the operation should happen, typically you identify a `Link` with a matching key. For that, you need a `find()` method that returns a pointer to the link. Because the `DoublyLinkedList` class is a subclass of the `LinkedList` and that has a `find()` method, as shown in [Listing 5-5](#), there's no need to rewrite one for

DoublyLinkedList. The methods for inserting after and deleting target Links are shown in [Listing 5-19](#).

Listing 5-19 *Methods to Insert and Delete Middle Links of DoublyLinkedLists*

```
def identity(x): return x          # Identity function
...
class DoublyLinkedList(LinkedList.LinkedList):
... (other definitions shown before) ...

    def insertAfter(                # Insert a new datum after the
        self, goal, newDatum,      # first Link with a matching key
        key=identity):
    link = self.find(goal, key)    # Find matching Link object
    if link is None:              # If not found,
        return False              # return failure
    if link.isLast():             # If matching Link is last,
        self.insertLast(newDatum) # then insert at end
    else:
        newLink = Link(           # Else build a new Link node with
            newDatum,              # the new datum that comes just
            previous=link,         # after the matching link and
            next=link.getNext())   # before the remaining list
        link.getNext().setPrevious( # Splice in reverse link
            newLink)              # from link after matching link
        link.setNext(newLink)     # Add newLink to list
    return True

    def delete(self, goal,         # Delete the first Link from the
        key=identity):           # list whose key matches the goal
    link = self.find(goal, key)   # Find matching Link object
    if link is None:             # If not found, raise exception
        raise Exception("Cannot find link to delete in list")
    if link.isLast():            # If matching Link is last,
        return self.deleteLast() # then delete from end
    elif link.isFirst():         # If matching Link is first,
        return self.deleteFirst() # then delete from front
    else:                        # Otherwise it's a middle link
        link.getNext().setPrevious( # Set next link's previous
            link.getPrevious())     # to link preceding the match
        link.getPrevious().setNext( # Set previous link's next
            link.getNext())         # to link following the match
    return link.getData()         # Return deleted data item
```

The `insertAfter()` method starts by using the `find()` method to find a `Link` whose key matches the given `goal`. If such a `Link` isn't found, then the method quits, returning `False` to indicate it failed. Alternatively, it could raise an exception or insert the data at the first or last position, depending on what's needed. After a `Link` with a `goal` key is found, `insertAfter()` checks whether it is the last link, and if so, uses `insertLast()` to place it after the last link.

After handling the special cases, `insertAfter()` can deal with the case of inserting between two consecutive links of the list. It builds a new `Link` holding the new data with the previous pointer set to the goal `Link`, and the next pointer set to the following `Link`. This is shown in the third panel of [Figure 5-13](#). The fourth panel shows how it alters the `__previous` pointer of the following `Link` and the `__next` pointer of the goal `Link` to point at the `newLink`. The order of these steps is very important.

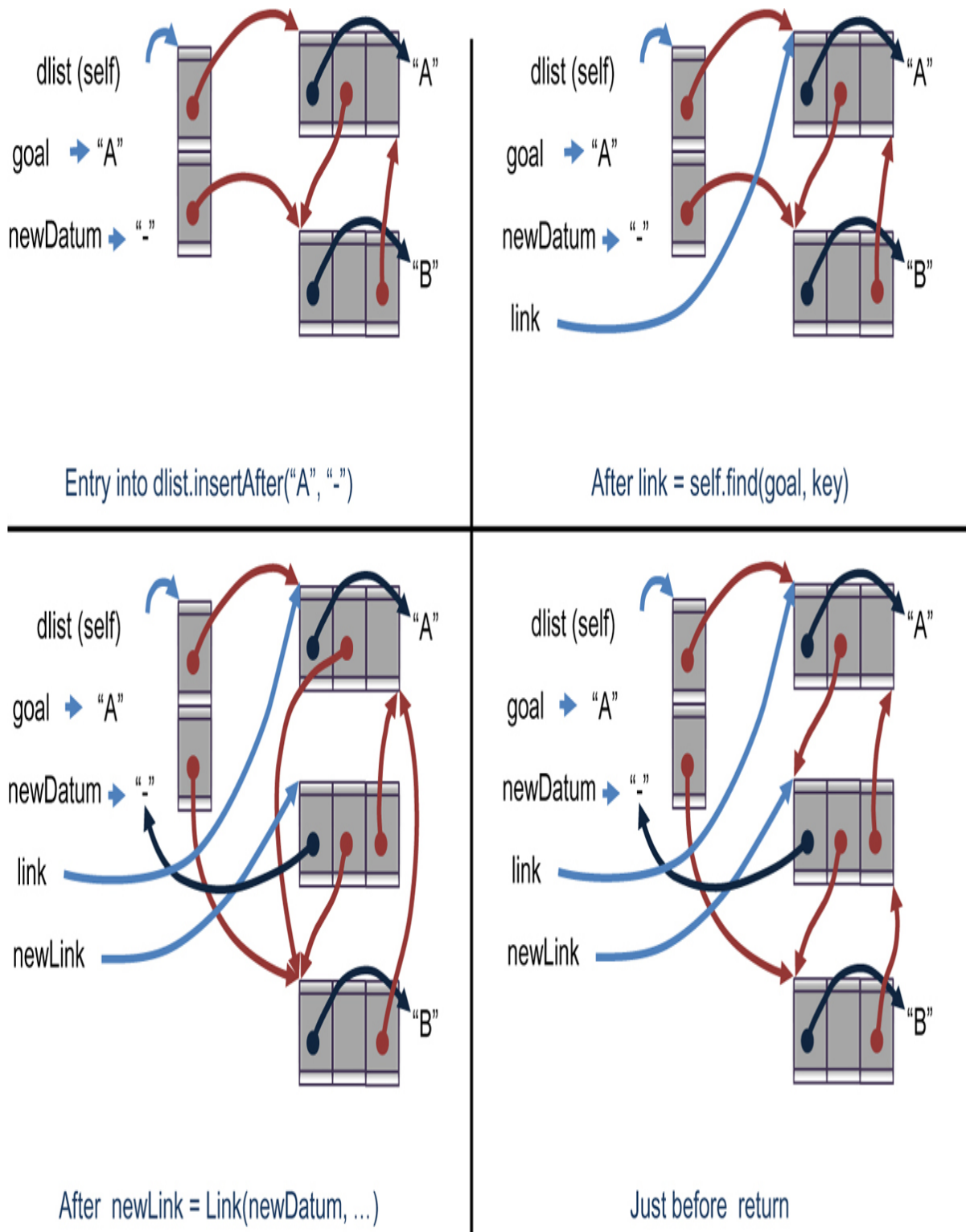


Figure 5-13 *Insertion in the middle of a doubly linked list*

The `delete()` method also starts by using the `find()` method to find a `Link` whose key matches the given `goal`. If such a `Link` isn't found, then it raises an

exception. Alternatively, it could return `False` to indicate it failed. Having found the `Link` to delete, it checks whether it's the last or first link of the list. If it is, it uses the deletion methods for the ends. Otherwise, it needs to “snip” out the `goal Link` by altering the `__previous` and `__next` pointers of the `Links` after and before it, respectively. With the pointers modified, it returns the data stored in the `goal link`.

Note that the doubly linked list avoids the need to search for the `Link` preceding the `goal` for both insertion and deletion. The data structure stores that pointer for every `Link`, unlike singly linked lists. Do you think that's a simplification in terms of how complicated it is to implement the insertion and deletion methods? Opinions differ on that, but remember that this kind of complexity is very different from the computational complexity of the algorithm. Complexity for the programmer is a completely separate concept from how efficiently the computer can perform the operations. Both are important for different reasons.

To verify the implementation of the doubly linked list, you can use a program like the `DoublyLinkedListClient.py` program shown in [Listing 5-20](#). Note that programs like this exercise the basic operations of a data structure but are not truly comprehensive test programs.

Listing 5-20 *The `DoublyLinkedListClient.py` Program*

```
from DoublyLinkedList import *

dlist = DoublyLinkedList()

for data in [(1968, 'Richard'), (1967, 'Maurice'), (1966, 'Alan')]:
    dlist.insertFirst(data)
for data in [(2015, 'Whitfield'), (2015, 'Martin'),
             (2016, 'Tim'),
             (2017, 'David'), (2017, 'John')]:
    dlist.insertLast(data)
print('After inserting', len(dlist),
      'entries into the doubly linked list, it contains:\n', dlist,
      'and empty =', dlist.isEmpty())

print('Traversing backwards through the list:')
dlist.traverseBackwards()

print('Deleting first entry returns:', dlist.deleteFirst())
```

```

print('Deleting last entry returns:', dlist.deleteLast())
def year(x): return x[0]
for date in [1967, 2015]:
    print('Deleting entry with key', date, 'returns',
          dlist.delete(date, key=year))
print('List after deletions contains:', dlist)

for date in [1968, 2015]:
    data = (date + 1, '?')
    print('Inserting', data, 'after', date, 'returns',
          dlist.insertAfter(date, data, key=year))
print('List after insertions contains:', dlist)

print('Traversing backwards through the list:')
dlist.traverseBackwards()

```

The program in [Listing 5-20](#) builds a doubly linked list holding pairs of a year and a name. It shows the contents of the list both in the string format, which steps through the list in the forward direction, and by calling the `traverseBackwards()` method to print each data item in reverse order. It tests deletion at both ends of the list, followed by deleting some entries by using the year as a key (it defines a `year()` function that returns the first entry of each pair). The items are printed as they are deleted, and the remaining list is printed after that. Then it inserts some new pairs after items with specific dates and prints the list both forward and backward to show that the linkages are preserved after all the changes. The output is

```

$ python3 DoublyLinkedListClient.py
After inserting 8 entries into the doubly linked list, it contains:
[(1966, 'Alan') > (1967, 'Maurice') > (1968, 'Richard') > (2015,
'Whitfield') > (2015, 'Martin') > (2016, 'Tim') > (2017, 'David') >
(2017,
'John')] and empty = False
Traversing backwards through the list:
(2017, 'John')
(2017, 'David')
(2016, 'Tim')
(2015, 'Martin')
(2015, 'Whitfield')
(1968, 'Richard')
(1967, 'Maurice')
(1966, 'Alan')
Deleting first entry returns: (1966, 'Alan')
Deleting last entry returns: (2017, 'John')
Deleting entry with key 1967 returns (1967, 'Maurice')

```

```

Deleting entry with key 2015 returns (2015, 'Whitfield')
List after deletions contains: [(1968, 'Richard') > (2015, 'Martin') >
(2016, 'Tim') > (2017, 'David')]
Inserting (1969, '?') after 1968 returns True
Inserting (2016, '?') after 2015 returns True
List after insertions contains: [(1968, 'Richard') > (1969, '?') >
(2015,
'Martin') > (2016, '?') > (2016, 'Tim') > (2017, 'David')]
Traversing backwards through the list:
(2017, 'David')
(2016, 'Tim')
(2016, '?')
(2015, 'Martin')
(1969, '?')
(1968, 'Richard')

```

Doubly Linked List as Basis for Deques

A doubly linked list can be used as the basis for a deque, mentioned in the preceding chapter. In a deque, you can insert and delete at either end, and the doubly linked list provides this capability. A programming project at the end of this chapter has you implement this ADT.

Circular Lists

All the lists we've looked at so far have a defined beginning and end. Occasionally, it's useful to create a chain of links that forms a circle or loop. For example, a chain of links could represent the players in a game where the turn passes from one player to the next and returns to the first when all players have had their turn. In some sports the players are organized in a rotation, taking turns at serving or batting. In chemistry there are chains of molecules that form rings. In biology there are organisms that depend on other organisms, and the dependencies sometimes form loops. The obvious question then becomes, "Where does the circular list end?" In each of these examples, there really isn't an end, but there's a need for a marker, or sometimes multiple markers, to designate a specific member of the list for various purposes.

Using a "circular" array to implement a queue is a convenient way to reuse array cells for storage after they had been inserted at one end and removed from the other. If the queue length was less than the size of the array, some cells of the array were not used. Circular lists that are implemented using

references between objects avoid having unused elements. That design saves space but introduces complications like determining the size of the list and traversing each element of the list exactly once. The double-ended list is a better design for a queue because it doesn't have unused elements and is simpler than circular lists. There can be other uses, however, where a circular list might help.

Figure 5-14 shows a typical way to represent circular lists. The list is singly linked, but it could be doubly linked if reversing the direction of traversal is important. It needs a marker for one of the elements to designate it as the last (or current or first), just like the `LinkedList` points at an individual `Link` object. The figure doesn't show any data stored in the links. Any data would be stored using references, the same way as the other linked lists we've discussed.

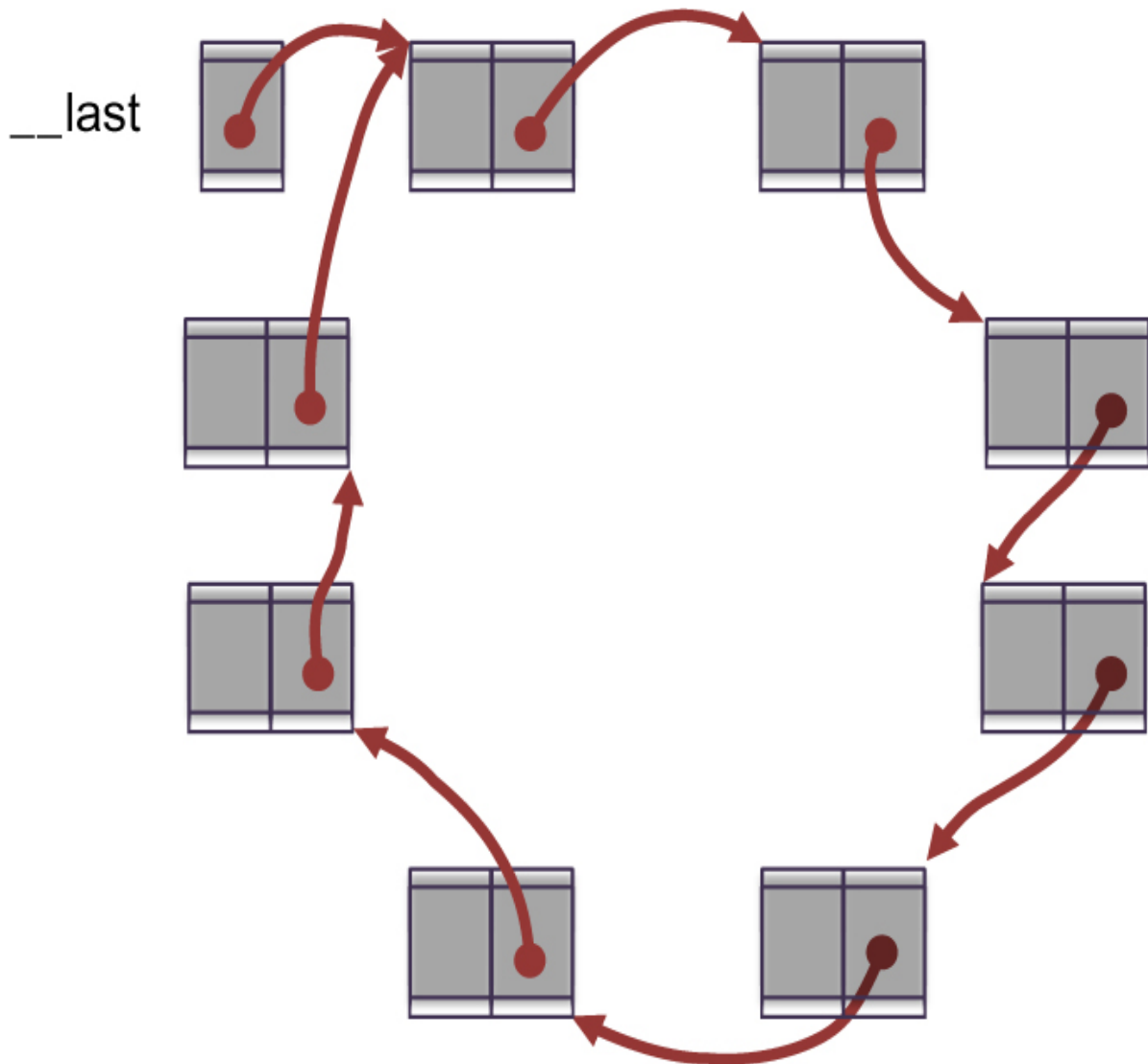


Figure 5-14 *A circular linked list (without data)*

If the only thing stored for the marker is a pointer to one of the `Link`s, then traversal requires following the pointers until the loop revisits the marked `Link`. That's no more complicated than following the pointers until `None` is found. By choosing to mark the last `Link` in the chain, you can find the first `Link` in two steps, making both `insert()` (or `insertFirst()`) and `insertLast()` methods constant time operations.

The biggest problem with circular lists is ensuring they remain circular. The individual `Link` objects would typically offer a `setNext()` method that would be used by the `CircularList` class to insert and delete links from the structure. If that `setNext()` method is public and callable by the code using the `CircularList` class, then the caller could set a link pointer to some other `Link`, or possibly set it to `None`, to form a noncircular, or linear, list. These can take unusual shapes, like the ones shown in [Figure 5-15](#).



Figure 5-15 *Broken circular lists (without data)*

Coping with anomalies in circular lists can be difficult. If `None` is allowed as a value for the next pointer, then the traversal loop conditions must look for that value in addition to finding the first visited link when determining the end of a traversal loop. Even that, however, won't help when the `__last` pointer is not pointing to a `Link` in a loop, as shown on the right in [Figure 5-15](#) where the `__last` field points at a "spur" off the loop. Traversal would start on the spur and progress onto the circular part. That means the loop condition must compare the current pointer to not only the first pointer but to *all* `Link`s visited so far. That's not much of a concern for a small circular list, but as the number of links grows large, it becomes very time-consuming. Each of the N steps in

the traversal must compare the current link with up to N other links, so the total number of comparisons is $O(N^2)$.

To avoid such problems, circular list implementations must avoid making the `setNext()` method public and offer other methods for altering the list that guarantee it remains a single loop with no spurs or breaks. Similarly, linked lists of all kinds (simply linked, doubly linked, double-ended) must not allow public `setNext()` methods to be used to create what is essentially a circularly linked list. If they do, then the traversal methods that terminate when `None` is found will never find it.

Iterators

All the linked lists and array structures you've studied have included a `traverse()` method. That method applies a function on every item stored in the data structure, in a known order. Visiting every item is very useful, for instance, when you need to update all the personnel records to assign a new identification code to every one of them or to print a mailing label for every record. The function passed to `traverse()` takes a single argument, an item stored in the list, and performs some operation.

Calling a function on every item works well when the operation to be performed on each item is independent of the operations performed on all the other items. On the other hand, operations that combine or have some interaction between the items are not as easy to handle with a single function on one item. For example, if you needed the average of some numeric field in each record, the function would need to collect the sum somewhere and then divide it by the number of items at the end. Another difficult example would be to collect pairs of records where some fields in each pair don't match one another.

For combining data across all the items in a data structure, it's more convenient to have some way to traverse the items in a loop inside the calling program. That allows the caller to keep track of sums, minimums, maximums, the last value processed, and so on. What's needed is a way to iterate over the items in the context of another program. Iterating this way is straightforward for arrays where the items are indexed by an integer and any item can be fetched in constant time. Data structures like lists and some of the more complex ones we study later don't have a way to access arbitrary items in constant time but can

follow a defined order to step through all the items with constant time access for each step.

An **iterator** is a data structure that provides methods to step through all the items contained in another data structure, visiting each item exactly once. The order of the items is fixed; each iterator for a particular object goes through the sequence of items in the same order as long as the contents of the object don't change. The iterator must provide at least one method, usually called `next()`, `getNext()`, or something similar, that is used to get the next item in the sequence. The other methods it provides are somewhat dependent on the data structure it steps through and the language used to implement it.

The object that the iterator steps through is sometimes called a **collection**, a **container**, a **sequence**, or an **iterable**. The choice of which term to use is somewhat dependent on the kind of data structure being sequenced. Python has sequence data types, such as `list` and `str`. There is a natural order of the items in those structures, so they are appropriately named. Other data structures like hash tables (dictionaries), trees, and sets have less obvious orderings. The items they contain, however, can still be put into a sequence that the iterator steps through.

There's another benefit to iterators: they can represent infinite sequences and sequences of indefinite length. For interactive programs, the various inputs coming in, like clicks, keystrokes, touches, sounds, camera images, and network packets, can be handled by iterators. It's usually unclear how many of these inputs will happen. The program that consumes them only needs to know that they come in order and that it should continue processing them until some termination code is received. In math, there are infinite sequences, such as all the integers or all the prime numbers. It's impractical to make a data structure that holds an infinite number of things, but in many cases, you can make one that steps through that infinite sequence in a particular order.

Python calls any class that can produce an iterator as *iterable*. We use that term in this book to describe any object being sequenced, although that term has a more specific meaning in Python. Let's examine how to construct iterators in general and then look at the specifics of how to implement them in Python.

Basic Iterator Methods

An iterator is a data structure in its own right that references another data structure. The iterator, however, is strongly dependent on the iterable, so they are frequently implemented together in the same source code. Because the iterator is a data structure, it is an object like other objects, and needs a constructor. Instead of defining a public class like `LinkedListIterator`, you often define the constructor for the iterator as a method of the iterable data structure. That constructor method could be called `iterator()`. For example, you could construct a linked list, fill it with some data, and create an iterator for it by executing

```
l1ist = LinkedList()
l1ist.insert(<some data>)

...
it = l1ist.iterator()
```

If you make a public class for `LinkedListIterator`, then the iterator would be constructed using something like

```
it = LinkedListIterator(l1ist)
```

The choice is mostly stylistic. The advantage of using a method in the iterable class is that it can be the same method name for every iterable class.

Programmers who use the data structure don't need to hunt for what the iterator class name is; they can just call `iterator()`.

After it is constructed, the caller gets the items by calling the iterator's `next()` method. The caller typically does this in a loop until all the items have been processed. Sometimes the iterator interface also has a method like `current()`. This is somewhat like the `peek()` method used in queues to look at the value at the front of the queue without removing it from the queue. Calling `next()` is like calling `pop()` from a stack; it changes the iterator to move on to the next item and returns an item. If the iterator has a `current()` method, then it must return the first item when the iterator is constructed (before any calls to `next()`).

Finishing the Iteration: Using a Marker or Sentinel Value

Because the iterator is likely to be used in a loop, it needs a good interface for handling the start and end of loops. If the iterable is empty, then the loop shouldn't be entered at all. If there are some items, then the loop must end after the iterator reaches the last one. If there is a known value that could never be

an item stored inside the iterable, then the loop condition could test for that. For example:

```
while it.current() is not None:
    <loop body>
    it.next()
```

This loop looks at the current value, and if it is not the marker for the end of the iteration (`None` in this example), it executes the loop body and advances to the next item. The question then becomes how to handle the end of a sequence where `current` could take on any value, including `None`.

Finishing the Iteration: Using a Termination Test

Another strategy for loop termination is to define a method like `hasMore()` that provides a Boolean value to say whether another call to `next()` will produce a value. That means the loop would look like this:

```
while it.hasMore():
    current = it.next()
    <loop body>
```

This termination test strategy eliminates the need for a known value as a marker for the end of the sequence. That allows storing `None`—or any other value—as one of the items in the list.

Finishing the Iteration: Using Exception Handling

Yet another strategy is to use exception handling. If the iterator has a `current()` method, then calling it or `next()` when the iterable is empty could cause an exception. If there is no `current()` method, then calling the `next()` method will cause an exception when it reaches the end of the sequence. We haven't discussed file data structures yet, but this is a common way to handle the end of a file or user input. In these cases, you need some exception handling around the iteration loop to detect the end of the sequence. For example, if the end of sequence causes a `StopIteration` exception to be thrown, then the iteration loop takes the form:

```
try:
    while True:
        current = it.next()
        <loop body>
```

```
except StopIteration:
    pass
```

This alternative is definitely longer in terms of number of lines of code. It's also a bit harder to follow the flow because the loop body is now two levels deep and there is an exception handler at the end. Before dismissing it based on that, however, note that it also shares the advantage that there is no special known value for the end of the sequence, and it doesn't require a `hasMore()` method to check for more items. The program can try to get another item from the iterable, perhaps pausing execution while waiting for new input to be added to the iterable, and either return the new item or throw an exception to say no more could be found.

Normally, `next()` returns the item stored in the object it sequences and not a reference to the internal structure that holds a reference to the item. For example, in a linked list, `next()` would return the item stored in the `__data` field of a `Link` object, and not a reference to the `Link` object itself. That's important for preserving the integrity of the references between links.

[Listing 5-21](#) shows a simple implementation for an iterator that handles two of these alternatives for termination. It iterates over the singly linked list (whose other definitions appeared in [Listings 5-3](#), [5-4](#), [5-5](#), and [5-6](#)). The iterator is a class of its own. In this case, we've chosen to make it a private class called `__ListIterator` that is *defined within* the `LinkedList` class. The rationale for keeping it private is that there should be no need to make subclasses or perform other operations on the iterator class.

Listing 5-21 *Simple Iterator for a `LinkedList`*

```
class LinkedList(object):

... (other definitions shown before) ...

class __ListIterator(object):    # Private iterator class
    def __init__(self, llist):    # Construct an iterator over a
        self._llist = llist      # linked list
        self._next = llist.getNext() # Start at first Link

    def next(self):               # Iterator's next() method
        if self._next is None:    # Check for end of list
```

```

        raise StopIteration    # At end, raise exception
    item = self._next.getData() # Store next data item
    self._next = self._next.getNext() # Advance to following
    return item

    def hasMore(self):          # Is there more to iterate?
        return self._next is not None # Check for end of list

    def iterator(self):
        return LinkedList.__ListIterator(self)

```

The implementation of the `__ListIterator` class has a constructor that records the linked list iterable in a field named `_l1st`. It initializes a second field called `_next` to point at the first `Link` of the linked list. If the list is empty, then `_next` will be `None`. The constructor is used by the `LinkedList`'s `iterator()` method at the end of [Listing 5-21](#) to create the iterator object. It's a little unusual in that it passes the `self` variable explicitly to the constructor; that approach is needed to distinguish the `LinkedList` object from the `__ListIterator` object.

The iterator class's `next()` method steps through the list and throws an exception at the end. The `next()` method uses the information stored in the iterator object to step through each of the items of the list. It first checks whether the internal `_next` pointer is `None`. If so, then either the iterator started on an empty list, or past calls to `next()` have advanced to the end of the list, and it throws the `StopIteration` exception. This is a predefined exception in Python, but it could just as easily be a user-defined exception class. If you choose this particular exception, it operates like Python's implementation of iterators, as discussed in the later [“Iterators in Python”](#) section.

When `_next` points to a `Link` object, then the `next()` method stores the item from that link; advances to the next link, which could be `None`; and returns the stored item. To use this iterator, a calling program would use a loop like this:

```

it = l1st.iterator()
print('Created an iterator', it)
try:
    while True:
        print('The next item is:', it.next())
except StopIteration:
    print('End of iterator')

```

Executing that program on a list of squares would produce something like this:

```
Created an iterator <LinkedList.__ListIterator object at 0x1100d5668>
The next item is: 16
The next item is: 9
The next item is: 4
The next item is: 1
The next item is: 0
End of iterator
```

The implementation of `__ListIterator` in [Listing 5-21](#) also defines a `hasMore()` method. Thus, the termination test strategy can be used by writing something like

```
it = llist.iterator()
print('Created an iterator', it)
while it.hasMore():
    print('The next item is:', it.next())
print('End of iterator')
```

This kind of multiple strategy iterator supports different programming styles.

Other Iterator Methods

The basic function of the iterator is to step through the sequence of items stored in another data structure, but it's sometimes convenient to add more capabilities. As mentioned earlier, examining the next item without calling `next()` (which advances the iterator to the following item) can be useful, such as when merging two ordered lists. You could build iterators for both lists, examine the first element from each one, and advance the iterator whose next item comes first in the sort order. A method named `getCurrent()` or `current()` or `peek()` is typically used for this purpose.

Another useful function is to point the iterator back at the beginning of the sequence. This might be used when identifying items that should get awarded labels in some priority order. Imagine handing out some delicious treats to the top five finishers of a race. One iterator could loop through the treats while another loops through the racers in the order they finished. Each racer is asked if they would like the treat currently being offered. If they accept, the treat iterator advances to the next treat, and the racer iterator resets back to the beginning. That way, any racer who finished faster gets priority for the next treat. (Of course, racers who already selected treats don't get to select again.)

For this kind of reset operation to work, the iterator must maintain a reference to the original iterable data structure. In the linked list example, that reference is stored in the `_l1ist` field, so a definition like

```
def reset(self):                                # Reset iterator to first link
    self._next = self._l1ist.getNext()
```

could be used to return to the beginning.

Altering Structures During Iteration

Some iterators provide methods to alter the iterable data structure. This operation is very tricky because changing the data structure could change the items it contains and/or the sequence those items should be visited. For example, if the last item from a list were to be deleted, presumably the iterator would skip over that item after iterating up to the second-to-last item. In the example of assigning treats to racers, it would be convenient to remove a racer from the iterator after they chose a treat. Taking them out of those subsequent iterations, however, is difficult to do.

It's somewhat easier to plan for such deletions by restricting them to be done only by the iterator so that any references to the iterable data structure can be updated appropriately. If alterations to the iterable happen somewhere else in the code, however, it is impossible for the iterator to properly handle all the conditions that might arise. For example, if, after creating an iterator on a linked list using the `iterator()` method in [Listing 5-21](#), some other part of the code deletes the first item in the list, what should happen? The internal `_next` pointer of the iterator points to a `Link` that is no longer part of the iterable `_l1ist`, and the subsequent call to `next()` would return an item that's not in the list. It's not clear where `next()` would advance the `_next` pointer to because the deleted `Link` could have had its pointer altered by the outside code.

When changes to the data structure are allowed while stepping through the iterable sequence, it's best that all changes to the sequence of items are performed through the iterator. Methods like `insertAfter()`, `insertBefore()`, and `deleteNext()` can be defined to change the list in the vicinity of the next item while still allowing for the rest of the sequence to be traversed. The `insertBefore()` method can be efficiently implemented if the list is doubly linked or the iterator maintains a reference to the previously visited `Link`. The `insertAfter()` method can be implemented with a singly linked list. There are two alternatives to handling the `_next` pointer after the insertion: include the

newly inserted item or skip over the newly inserted item in the remaining sequence of the iterator. If the `_next` pointer is not advanced, then multiple insertions could add several items that would then be visited in the reverse order of their insertion. If the `_next` pointer is advanced, then multiple items could be inserted and would retain the order of insertion (and be skipped by the iterator sequence). The `deleteNext()` method would remove an item from the iterable and from the iterator sequence.

As you can see, mixing iteration over an iterable sequence with alterations to the iterable can cause horrendous problems. It's best to prevent alterations while iterating.

Iterators in Python

Python makes traversing data structures very easy with special support for iterators. In fact, all the `for x in [a, b, c]:` style loops you've been using are implemented using iterators. Much of the detail described previously for implementing iterators is performed by Python without any visible (user) source code. It's important to learn about how that works as you implement data structures in other languages with different syntaxes.

As you might expect, the built-in `list` type is iterable. Strings and tuples are also iterable. The general syntax for a loop using an iterator is

```
for var in <iterable>:  
    <body>
```

Looking under the hood, the way that Python executes a loop in this form is to evaluate the `<iterable>` expression on entry into the loop. The `<iterable>` expression produces an iterator object that is not directly available to the program but is maintained by the Python interpreter. The interpreter then creates a new local variable, `var`, and assigns it a value by calling the `next()` method of the iterator. More precisely, the iterator is created by calling the `__iter__()` method of the object returned by evaluating the `<iterable>` expression.

With the iterator created, the interpreter then calls its `__next__()` method to get the next item in the sequence. After the variable is assigned, it evaluates the loop body. When the body finishes, the interpreter calls the `__next__()` method to obtain the next item in the sequence. When the iterator goes past the last item in the data structure, the `__next__()` method raises the

`StopIteration` exception. That exception could happen on the first attempt to call the `__next__()` method, in which case the variable is not assigned, and the loop body is not evaluated at all.

To be a little more concrete, let's look at an example where *var* is *v* and the *<iterable>* expression is *expr*. The Python interpreter essentially executes the following:

```
it = expr.__iter__()
try:
    while True:
        v = it.next()
        <body>

except StopIteration:
    pass
```

This isn't exactly the translation of the `for var in <iterable>: loop`; the *it* variable is hidden, and the *v* variable is accessible only in the loop *<body>*, just like local variables are accessible only within a function definition. The rewritten version shows, however, the basic steps the interpreter takes. The `__iter__()` method creates the iterator object, *it*, and its `__next__()` method is called in each pass through the loop. The loop creates a hidden exception handler for `StopIteration`. When that exception occurs, the “infinite” `while` loop stops, and execution passes on to whatever comes after the loop.

If any other exceptions happen in the loop body, they are ignored by the hidden exception handler and passed on to the next containing level of the program. If there are nested loops in the *<body>*, they create their own exception handlers that catch the end of the iteration for each of those loops without affecting the end of this outer loop. Because the iterator and the exception handler that catches its termination are not directly accessible in the source code, simple Python loops provide an *implicit iterator* rather than an *explicit* one like in [Listing 5-21](#).

Generators

Any Python object class that implements an `__iter__()` method becomes an iterable object. The iterator object it returns must implement both a `__next__()` and an `__iter__()` method. These methods could be written using a hidden class like the one shown in [Listing 5-21](#), but there's an easier way: **generators**. Generators are functions that create iterators. Python has a special

statement, the `yield` statement, that takes care of creating the iterator class and its required methods. Let's look at an example using a mathematical sequence.

One of the most famous infinite sequences is the Fibonacci sequence. It occurs in nature in the fruit sprouts of a pineapple, flowers of an artichoke, and bracts (petals) of pinecones. It starts with the number 1, repeated twice. Each subsequent number in the sequence is the sum of the previous two numbers. The sequence begins

1, 1, 2, 3, 5, 8, 13, 21, ...

You can write a function that prints out all the Fibonacci numbers like this:

```
def Fibonacci():
    previous = 0
    current = 1
    while True:
        print(current)
        next = previous + current
        previous = current
        current = next
```

This function never ends because it prints out each number in the sequence (so be careful when executing it). You can convert this function into a generator with a simple change, replacing the `print` statement with a `yield` statement, as shown in [Listing 5-22](#).

Listing 5-22 *A Python Generator for the Fibonacci Sequence*

```
def Fibonacci():
    previous = 0
    current = 1
    while True:
        yield current
        next = previous + current
        previous = current
        current = next
```

The `yield` statement is a little bit like the `return` statement of a function. It causes the execution of the function to stop and return the value of the expression to its caller. What's very different from the `return` statement is that the Python interpreter preserves the state of execution of the function so that

the next time it is called, execution resumes just after the `yield` statement and continues the processing. Continuing the processing makes the `yield` statement act more like the `print` statement it replaced. You might think of `yield` as sending a message back to the caller without losing track of what happens next.

When the Python interpreter finds the `yield` statement inside the definition of `Fibonacci()` in [Listing 5-22](#), it changes the defined function to a special form called a *generator function*. The presence of `yield` means that this function shouldn't execute like most functions. When the generator function is called, it returns a generator object, which is a kind of iterator. That means you can use it to step through the sequence using `__next__()`. You can try it out in the Python interpreter using the definition in [Listing 5-22](#):

```
>>> gen = Fibonacci()
>>> gen
<generator object Fibonacci at 0x1012386d0>
>>> gen.__next__()
1
>>> gen.__next__()
1
>>> gen.__next__()
2
>>> gen.__next__()
3
```

The call to `Fibonacci()` produces an object that is stored in the `gen` variable. That object is a generator object. Because it's an iterator as well, it must have a `__next__()` method. The first call to `__next__()` produces (yields) the first number in the sequence, 1. Inside the interpreter, that first call starts the execution of the body of the `Fibonacci()` definition up to the first `yield` statement. That operation yields the value of `current`, which is initially 1. The second call to `__next__()` yields 1 again after the execution of the body resumes just after the `yield` statement. Let's look more carefully at how that happens.

After yielding the first 1, the second call to `__next__()` causes execution to resume in `Fibonacci()` after the `yield` statement. The `current` and `previous` variables have the same values they had during the first call, 0 and 1, because they are stored in the generator object. The next step in executing the loop body stores a new value, 1, in the `next` variable. Then it updates the `previous` and `current` values, returns to the top of the loop, and hits the `yield` statement again. The third call to `__next__()` yields 2 as the process repeats. The

generator object always keeps the values of `next`, `current`, and `previous` stored as part of the state of execution of the body so that it can resume right after the `yield` statement.

Having defined `Fibonacci()` as a generator, you can use it where iterable objects are expected like loops. The following example demonstrates that with a loop and a counter to print the first 15 numbers in the Fibonacci sequence:

```
count = 15
print('The first', count, 'numbers in the Fibonacci series are:')
for x in Fibonacci():
    if count < 1:
        break
    print(x)
    count -= 1
```

Note that the variable `x` is not bound to the value of the call to `Fibonacci()`. That generator object gets stored in some inaccessible location. Instead, the variable `x` gets bound to the values yielded by the successive calls to `__next__()`. The program outputs

```
The first 15 numbers in the Fibonacci series are:
1
1
2
3
5
8
13
21
34
55
89
144
233
377
610
```

This generator object could keep on producing Fibonacci sequence numbers indefinitely, so it essentially represents the infinite series in a finite data structure! The definition of `Fibonacci()` has a `while True` loop in it. That's what makes it go on forever. Using the `break` statement and decrementing the `count` prevent an infinite loop. This can be a powerful combination.

Python accomplishes this feat by creating an iterator data structure that holds the body of the function definition, the environment within that function (the state of all its parameters and local variables), and a pointer to where execution should resume in that function when `__next__()` is called. These combined “housekeeping” features makes it easy to write generators that will implement the iterators for data structures.

[Listing 5-23](#) shows one way to create the iterator for the singly linked list data structure using the `yield` statement in Python. It defines an `__iter__()` method that looks a lot like the `traverse()` method ([Listing 5-4](#)). Compare this definition of `__iter__()` to the `iterator()` method in [Listing 5-21](#), which involved defining an internal class for the iterator.

Listing 5-23 *Using a Python Generator to Make the `LinkedList` Iterator*

```
class LinkedList(object):

... (other definitions shown before) ...

    def __iter__(self):                # Define an iterator for the list
        next = self.getFirst()        # Start with first Link
        while next is not None:       # As long as the link is not None,
            yield next.getData()      # yield data for the link
            next = next.getNext()     # then move on to next link
```

Defining the `__iter__()` method as a generator function instead of a regular function takes care of building the extra class needed for an iterator. The iterator still needs to start with a pointer to the first `Link`, stored in its local `next` variable, and follows the chain of references until a `None` is found. For valid links, the iterator yields the data and moves on to the next link. When there are no more links, the loop ends, and the generator exits, raising the `StopIteration` exception. Then when the generator is invoked by a loop like

```
for item in myList:
    print(item)
```

Python catches the `StopIteration` exception with an implicit handler to break out of the `for` loop after printing each `item`.

It might seem a little unsettling that changing `return` statements to `yield` statements in a Python function has such a big effect on the behavior of the

code. When you use generators, a lot of work is done by the compiler without any explicit variables or control statements in the source code. When you're reading code that someone else wrote, it won't be clear whether something is a subroutine, function, or generator until you locate a specific `return` or `yield` statement. If you find a `return`, then it's a function, possibly with explicit values to be returned. If you find `yield`, then it's a generator that will first return an iterator.

Every iterator can produce a sequence of values. When the iterator finishes the sequence (assuming it has an end), it raises a `StopIteration` exception. Note the source code of the generator doesn't have an explicit `raise StopIteration` statement. The exception is raised sort of like the implicit `return None` when execution reaches the end of the body of a function. In fact, in Python 3.7 and later, a generator that explicitly raises a `StopIteration` exception causes a `RuntimeError`. Instead, you should use the implicit end of a function or explicit `return` with no argument to signal the end of the sequence.

Using `yield` statements is a compact way to define iterators, although the special behavior of the generator function can be a little hard to understand initially. The Programming Projects in this chapter give you chance to practice writing some.

Summary

- A linked list consists of one `LinkedList` object and a number of `Link` objects.
- The `LinkedList` object contains a reference to the first link in the list.
- Each `Link` object contains two things: data and a reference to the next link in the list. The data could be the data itself (like integers, characters, or bytes) or some reference to another, possibly larger, structure.
- A special value (typically `None` in Python) in the reference to the first or next `Link` signals the end of the list.
- Inserting an item at the beginning of a linked list involves creating a new `Link` with its data field holding or pointing to the item, changing the new `Link`'s next field to point to the old first link, and changing the `LinkedList`'s first field to point to the new `Link`.

- Deleting an item at the beginning of a nonempty list involves setting the first field to point to the first `Link`'s `next` field.
- To traverse a linked list, you start at first `Link` and then go from link to link, using each link's `next` field to find the next link.
- Some implementations use the same named field or accessor method for the first reference of the `LinkedList` as is used for the next reference of each `Link`. This makes the traversal code simpler.
- A link with a specified key value can be found by traversing the list.
- A new link can be inserted before or after a link with a specified key value, following a traversal to find this link.
- A double-ended list maintains a pointer to the last link in the list as well as to the first.
- A double-ended list allows insertion at the end of the list in constant time rather than $O(N)$ time for a singly linked list.
- An abstract data type (ADT) is a data storage class considered without reference to its implementation.
- Stacks and queues are ADTs. They can be implemented using either arrays or linked lists.
- In an ordered linked list, the links are arranged in order of ascending (or sometimes descending) key value.
- Insertion in an ordered list takes $O(N)$ time because the correct insertion point must be found. Deletion of the smallest (or sometimes largest) link takes $O(1)$ time.
- In a doubly linked list, each link contains a reference to the previous link as well as the next link.
- A doubly linked list permits backward traversal and deletion from the end of the list.
- Circular lists are similar to singly linked lists, but there is no last item. Instead of having a next pointer set to `None` for the last item, that item points back to the first `Link`.

- Circular lists have one item designated as a marked item. It can be called first, last, head, tail, mark, and so on. This is the `Link` pointed to by the circular list object.
- Singly linked circular lists have the same efficiency as linear linked lists. By marking the last item instead of the first, the `insert()` and `insertLast()` methods on a circular list take $O(1)$ time instead of $O(N)$.
- Circular lists must be carefully updated to preserve the single loop structure to avoid making it difficult to terminate loops through all the items.
- An iterator is a reference, encapsulated in a class object, that points to one item in another data structure. When the data structure is a linked list, the iterator references a particular link in that list.
- Iterator methods allow the calling program to move the iterator sequentially through all the items of the data structure. For linked lists, this follows the next pointers in each link.
- The data structure being sequenced by the iterator is called an iterable or a container.
- Iterators allow access to the data stored in the iterable without direct access to iterable's structures and references (`Links` in a linked list) to avoid allowing the calling program to alter the internal structure.
- An iterator can be used to traverse through a list, performing some operation on selected links (or all links).
- If the iterable changes while an iterator is stepping through its items, the iteration sequence can be altered, sometimes to the point of causing bad errors.

Questions

These questions are intended as a self-test for readers. Answers may be found in [Appendix C](#).

1. Which of the following is **not** true? A reference to a class object
 - a. can be stored in the data field of a singly linked list.

- b. can be used to access public methods in the object.
 - c. has a size dependent on its class.
 - d. does not hold the object itself.
2. Access to the links in a linked list is usually through the _____ link.
3. Lists differ from arrays in that
- a. lists have a fixed size per item, whereas arrays don't.
 - b. the relationships between items are explicit in lists but not in arrays.
 - c. position is only explicit in arrays; the key of a list item determines its position.
 - d. to get the Nth item in a list, a program must follow N links, whereas in an array, the program can compute the position of the item from N.
4. How many references must be set or changed to insert a link in the middle of a singly linked list?
5. How many references must be set or changed to insert a link at the end of a singly linked list?
6. In the `insert()` method in the `LinkedList.py` program, Listing 5-5, the statement `link = Link(datum, self.getNext())` means that
- a. the `datum` will be assigned to the `__data` field in the first `Link` of the `LinkedList`.
 - b. the `__next` field of the new link will refer to the `LinkedList`'s first `Link`.
 - c. `link` will be set to the first link whose data matches the given `datum`.
 - d. a new, two-item linked list with the `datum` and the next link will be stored in `link`.
7. Assume you are writing a method of `LinkedList` and the variable `x` holds the next-to-last `Link`. What statement will delete the last link from the `LinkedList`?
8. When one function calls another function using a variable, as in `do_something(x)`, when can the value of `x` in the calling function be changed by the called function?

9. A double-ended list
 - a. allows inserts to be performed at either end in constant time.
 - b. has its last link connected to its first link.
 - c. is another name for a doubly linked list.
 - d. has pointers running both forward and backward between links.
10. An abstract data type
 - a. specifies the fields of the data type without defining any methods.
 - b. allows references to data to be stored instead of the data itself.
 - c. defines the kinds of data that are represented and the operations that can be performed on them.
 - d. is used as a placeholder for a data collection inside a large program while other modules are written.
11. Consider storing an unordered collection of records in an array or in a linked list. Assuming copying a record takes longer than comparing keys to find a record, is it faster to delete a record with a certain key from a linked list or from an array?
12. How many times would you need to traverse an unordered doubly linked list to delete the item with the largest key?
13. Of the lists discussed in this chapter, which one would be best for implementing a queue?
14. Of the lists discussed in this chapter, which one would be best for implementing a priority queue?
15. Which of the following is **not** true? Iterators could be useful if you wanted to
 - a. do an insertion sort on a linked list.
 - b. delete all links with a certain key value.
 - c. swap two items with the keys A and B in a list.
 - d. insert a new link at the beginning of a list.
16. Which do you think would be a better choice to implement a stack: a singly linked list or an array?